

1.1 CORE DEFINITIONS

- Tree:**
- A hierarchical structure of nodes.
 - No cycles; connected graph.
 - One root node; child nodes.
- Binary Tree:**
- Each node has at most 2 children: Left and Right.
- Binary Search Tree (BST):**
- Left Subtree < Parent < Right Subtree.
 - Inorder traversal yields sorted order.

1.2 TYPES OF BINARY TREES

- Full Binary Tree:**
- Every node has 0 or 2 children.
- Complete Binary Tree:**
- All levels filled except possibly the last, which is filled left-to-right.
- Perfect Binary Tree:**
- All leaf nodes at the same level.
- Balanced Binary Tree:**
- Height diff between left/right subtree ≤ 1 for all nodes.

1.3 TERMINOLOGY

- **Node:** Basic unit.
- **Root:** Topmost node.
- **Leaf:** Node with no children.
- **Height:** Number of edges on longest path to leaf.
- **Depth:** Number of edges to root.
- **Subtree:** Node and all descendants.

DEPTH FIRST SEARCH (DFS)

1. Preorder (Root, Left, Right)

```
void preorder(TreeNode root) {
    if (root == null) return;
    print(root.val);
    preorder(root.left);
    preorder(root.right);
}
```

2. Inorder (Left, Root, Right)

```
void inorder(TreeNode root) {
    if (root == null) return;
    inorder(root.left);
    print(root.val);
    inorder(root.right);
}
```

BFS / LEVEL ORDER TRAVERSAL

Breadth First Search (BFS) Template

```
Queue q = new LinkedList<>();
q.offer(root);
while (!q.isEmpty()) {
    int size = q.size();
    for (int i = 0; i < size; i++) {
        TreeNode curr = q.poll();
        if (curr.left != null) q.offer(curr.left);
        if (curr.right != null) q.offer(curr.right);
    }
}
```